

C++ Code Generation Model (Custom LLM) — Project Documentation

Overview

This project aims to build a small language model capable of understanding and generating C++ code. The model is trained from scratch using a custom tokenizer and a Transformer architecture. The dataset is collected from open-source C++ repositories.

The long-term goal of the project is to create a system capable of:

- Generating C++ code
- Explaining physics equations
- Helping build simulations such as black hole or N-body simulations

This document explains every step completed so far in the project pipeline.

Project Pipeline

The complete pipeline currently looks like this:

GitHub Repositories ↓ C++ Source Files ↓ Dataset Extraction ↓ Dataset Cleaning ↓ BPE Tokenizer Training
↓ Dataset Tokenization ↓ Token ID Dataset (tokens.npy)

The model training step will come next.

Step 1 — Dataset Collection

The first step was collecting a large dataset of C++ code from GitHub repositories.

The following types of repositories were included:

1. Physics and simulation repositories
2. Ray tracing and graphics engines
3. Large production C++ libraries
4. Competitive programming repositories

Examples include:

- N-body simulation repositories

- Bullet physics related projects
- Competitive programming libraries
- Algorithm collections

These repositories were cloned locally using git.

Example commands used:

```
mkdir repos cd repos
```

```
git clone <repo_url>
```

This produced a local folder containing thousands of C++ files.

Step 2 — Extracting C++ Files

After cloning repositories, we extracted only relevant C++ source files.

File extensions used:

- .cpp
- .cc
- .cxx
- .h
- .hpp

A Python script recursively scanned all repositories and collected these files.

Each file's contents were appended into a single dataset file:

```
cpp_dataset.txt
```

To separate files inside the dataset we inserted markers such as:

```
FILE_SEPARATOR
```

This produced a large text dataset containing all C++ code.

Final result:

Approximately 6873 C++ source files combined into one dataset.

Step 3 — Dataset Cleaning

Many open-source repositories contain large license headers at the beginning of files.

Example:

```
/ Copyright (c) ... Permission is granted ... /
```

These license blocks repeat thousands of times and do not help the model learn programming patterns.

Therefore a cleaning script was used to remove large multi-line comment blocks.

The cleaning process removed:

- Large license headers
- Repeated boilerplate comments

But preserved useful comments such as:

```
// compute gravitational force
```

The cleaned dataset was saved as:

```
cpp_dataset_clean.txt
```

Step 4 — Building a Custom Tokenizer

A tokenizer converts raw text into numerical tokens that a neural network can understand.

The tokenizer used here is a Byte Pair Encoding (BPE) tokenizer.

Why Tokenization Is Required

Neural networks cannot process raw text.

Instead they process sequences of numbers called token IDs.

Example:

C++ code:

```
std::vector<int>
```

Possible tokens:

[712, 55, 980, 44]

Each number represents a token in the vocabulary.

Step 5 — How BPE Tokenization Works

The tokenizer starts with basic byte tokens (0–255).

Then it repeatedly merges the most frequent pairs of tokens.

Example process:

Initial tokens:

s t d ::

Possible merges:

s + t → st

st + d → std

• : → ::

These merges allow the tokenizer to learn common programming patterns such as:

- std
- ::
- ->
- vector

The tokenizer was trained on the cleaned dataset.

Vocabulary size chosen:

3000 tokens

This means:

256 base byte tokens + ~2744 merge operations

The tokenizer training produced a file:

tokenizer.json

This file stores:

- vocabulary
 - merge rules
-

Step 6 — Encoding the Dataset

Once the tokenizer is trained, the dataset must be converted into token IDs.

This step transforms:

cpp_dataset_clean.txt

into

numerical tokens.

Example transformation:

Code:

```
int main() { return 0; }
```

Tokens:

```
[421, 88, 912, 45, 67, 12]
```

These numbers are what the neural network will train on.

Step 7 — Saving Tokens

The encoded tokens are saved as a NumPy array file:

tokens.npy

This file contains a long sequence of token IDs.

Example:

```
[421, 88, 912, 45, 67, 12, 305, 918, ...]
```

Saving the dataset this way has major benefits:

- much faster loading
 - efficient storage
 - optimized for machine learning
-

Why Pre-Tokenizing the Dataset Is Important

Training a model involves millions of training steps.

If the dataset were tokenized during every training step, training would become extremely slow.

Instead the dataset is tokenized once and stored in `tokens.npy`.

The training process then loads token IDs directly.

Current Project Status

Completed components:

Dataset collection Dataset extraction Dataset cleaning Tokenizer implementation Tokenizer training Dataset encoding

Current outputs:

`cpp_dataset_clean.txt` `tokenizer.json` `tokens.npy`

Next Step (Upcoming)

The next stage of the project is training a Transformer model.

The model will be trained using next-token prediction.

Example:

Input tokens:

```
std :: vector < int > v ;
```

Target prediction:

push_back

By learning these patterns, the model will eventually generate valid C++ code.

Long-Term Goal

The final system aims to support:

- C++ code generation
- algorithm implementation
- simulation code assistance
- physics equation explanation

Eventually the model could help generate code for scientific simulations such as black hole physics or N-body gravitational systems.

Author Notes

This project intentionally builds core components from scratch rather than relying entirely on existing frameworks. The purpose is to understand the internal mechanics of language models including tokenization, dataset preparation, and transformer training.